

Augmix : A simple data processing method to improve robustness and uncertainty

고성혁 / '26. 4. 28(화)

관련문서	ICLR 2020, Augmix : A simple data processing method to improve robustness and uncertainty
논의사항 및 의문사항	
기타사항	



Augmix : A simple data processing method to improve robustness and uncertainty(ICLR, 2020)

고성혁 / '26. 4. 28(화)



Computational Data Science LAB



CONTENTS

1. Introduction

2. Augmix

3. Experiment



4. Conclusion

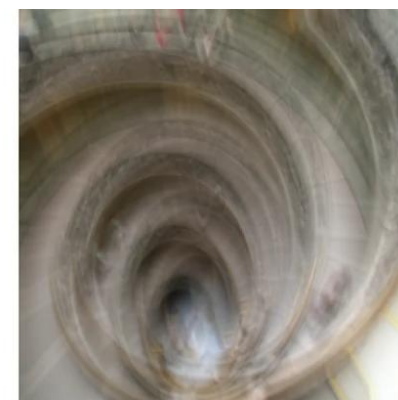
01 | Introduction

데이터 변화에 대한 모델의 취약성

- Data Shift – 학습데이터와 실제 데이터 분포가 다른 경우
- 이런경우 모델이 validation, test 집합에서 높은 성능을 보였음에도 일반화 성능이 떨어짐
- 실제 상황에서 굉장히 자주 발생하는 문제지만 연구가 많이 되고 있지 않음.
 - ✓ Unseen corruption(ex. Blur, noise)가 등장하는 경우
 - ✓ Label의 비율이 1:9였던 학습, 검증 데이터가 실제상황에서는 5:5의 비율인 경우
- 이러한 데이터 Shift 상황에서 모델 성능이 크게 저하됨
 - ✓ 실제환경에서는 예시와 같이 다양한 변형이 발생→ 성능저하
 - ✓ 따라서 Robustness 향상이 크게 중요함.



Ground Truth



Blurry

01 | Introduction

Robustness under Data Shift

- Corruption Robustness를 확인하기 위한 Datasets : ImageNet-C, CIFAR10-C, CIFAR100-C *corruption : 이미지에 가해지는 다양한 변형(noise, blur, weather)
- Test data에 대해 다양한 corruption을 여러 강도(severity)로 적용
- 모델이 학습에서 못 본(Unseen) corruption에 잘 대처하는지 평가
 - ✓ 학습과정에서는 해당 corruption을 사용하지 않음.



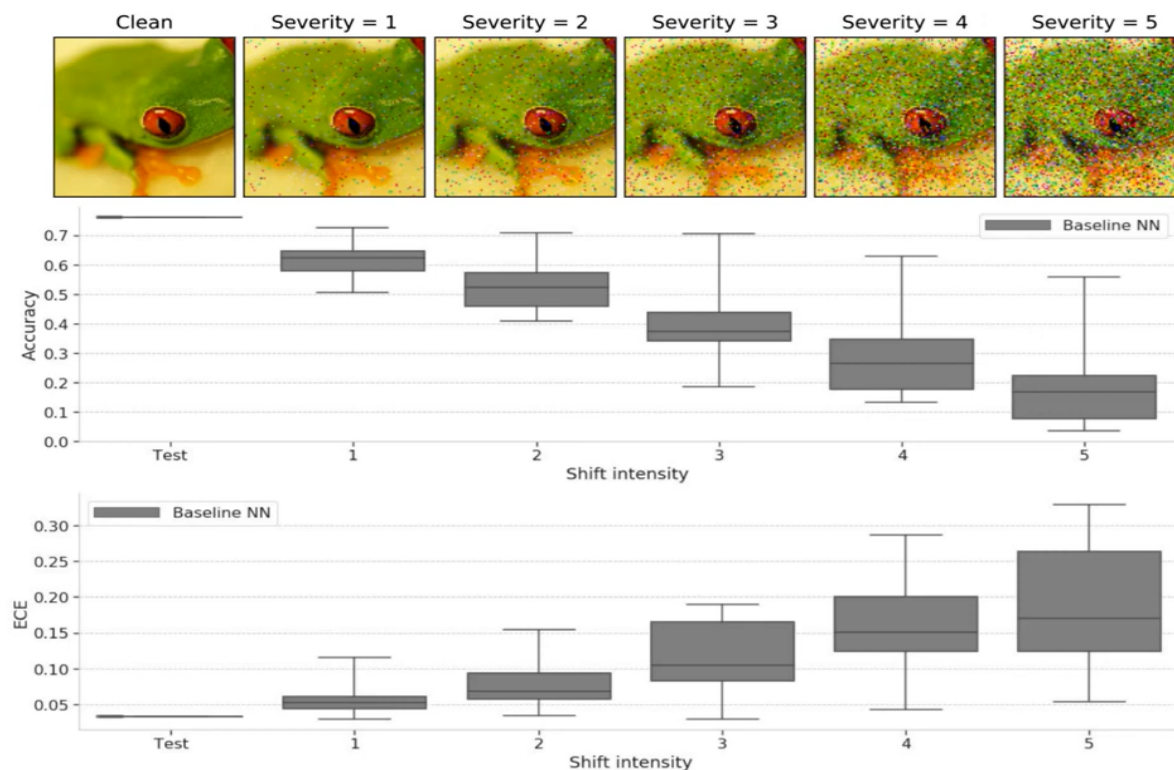
ImageNet-C

01 | Introduction

Effect of Corruption

- Clean Data로 학습된 Baseline 모델에 ImageNet-C를 Test한 결과

- Accuracy drops** with increasing shift on Imagenet-C
- Quality of uncertainty degrades** with shift -> “overconfident mistakes”



NeurIPS 2020 OOD Tutorial

[Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift](#)



01 | Introduction

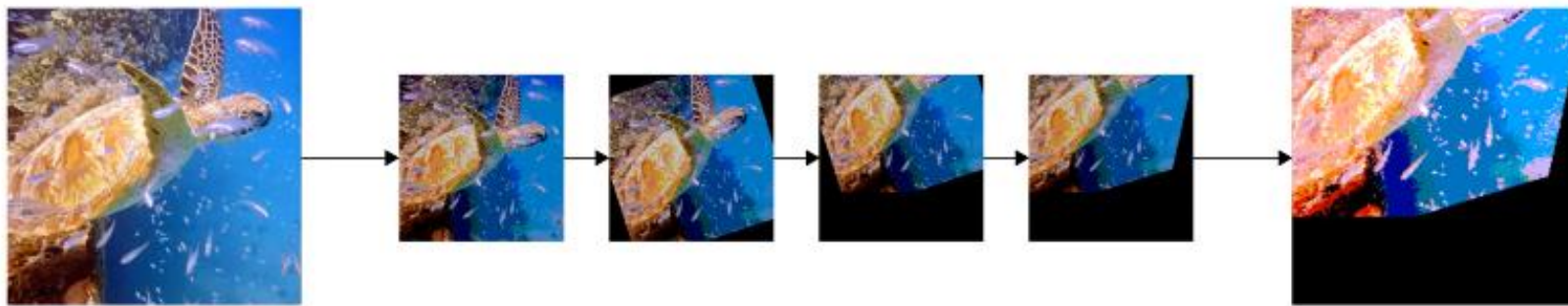
Limitations of Existing Approaches

- Data Augmentation 기반 방법
 - ✓ 특정 transformation에 의존하여 Unseen corruption에 일반화 어려움
 - ✓ 다양한 변형을 충분히 반영하기 어려움
- Adversarial Training 기반 방법
 - ✓ 계산 비용이 매우 큼
 - ✓ 특정 perturbation에 과적합 될 가능성 존재
- Robustness 관점의 한계
 - ✓ 다양한 corruption에 대해 일관된 성능 확보가 어려움
 - ✓ Accuracy와 uncertainty를 동시에 개선하기 어려움 → 보다 일반적인 robustness 방법 필요

02 | Augmix

Augment and Mix

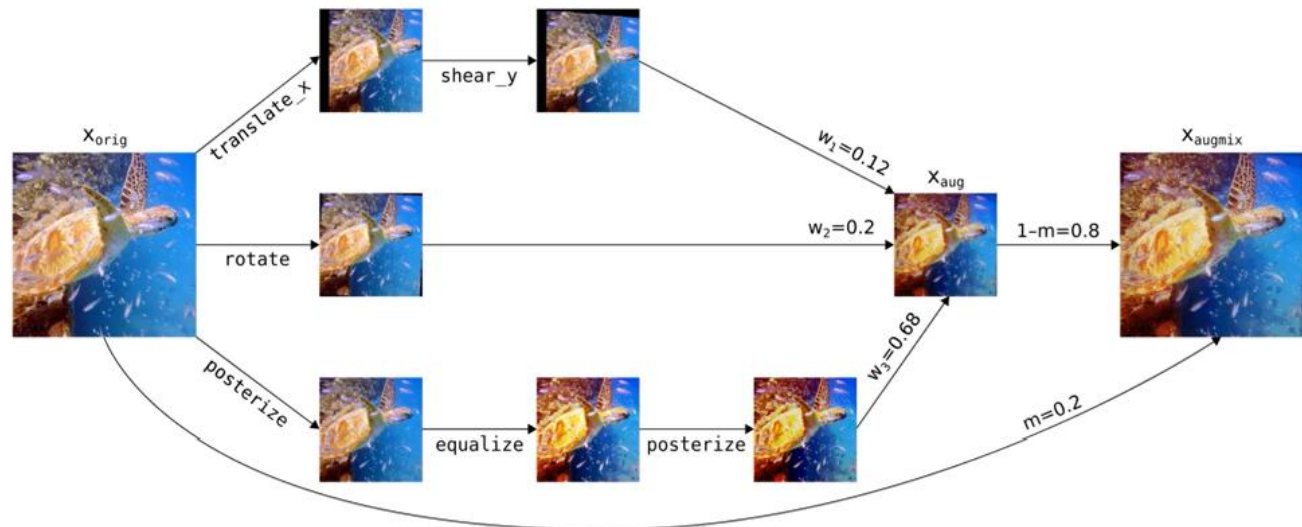
- 다양한 augmentation 생성 (augmentation chain) 여러 결과를 Mixing (weighted sum)후 원본이미지와 다시 결합 * Chain = augmentation 여러 개 이어 붙인 것 (원본→rotate→color→blur)
- Random Augmentation : aug종류도 랜덤, 순서도 랜덤, 길이(몇 개 이어 붙일지) 랜덤



- 여러 Aug Chain을 생성한 뒤 각 결과를 Convex Combination으로 Mixing(다양한 변형)
 - ✓ Convex Combination – 여러 이미지를 서로 다른 weight로 섞는 것 (weight 의 합 =1)

02 | Augmix

Augment and Mix



Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} += w_i \cdot \text{chain}(x_{\text{orig}})$   $\triangleright$  Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:  Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:  return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix}1} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix}1}$  is stochastically generated
16:  $x_{\text{augmix}2} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix}1} \neq x_{\text{augmix}2}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(y | x_{\text{orig}}), y) + \lambda \text{Jensen-Shannon}(\hat{p}(y | x_{\text{orig}}); \hat{p}(y | x_{\text{augmix}1}); \hat{p}(y | x_{\text{augmix}2}))$ 

```

02 | Augmix

Augment and Mix



- Dirichlet 분포
- 여러 비율을 랜덤으로 만들되, 합이 1이 되게 해주는 것"

Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} \mathrel{+}= w_i \cdot \text{chain}(x_{\text{orig}})$   $\triangleright$  Addition is elementwise
10:   end for
11:   Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:   Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:   return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix}1} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix}1}$  is stochastically generated
16:  $x_{\text{augmix}2} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix}1} \neq x_{\text{augmix}2}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(y | x_{\text{orig}}), y) + \lambda \text{Jensen-Shannon}(\hat{p}(y | x_{\text{orig}}); \hat{p}(y | x_{\text{augmix}1}); \hat{p}(y | x_{\text{augmix}2}))$ 

```

1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
 $\rightarrow \alpha = 10$ 이므로 uniform하게 원소의 합이 1이 되는 3차원 벡터가 샘플링 됨

02 | Augmix

Augment and Mix



Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{orig}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{orig}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{aug}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $op_1, op_2, op_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $op_{12} = op_2 \circ op_1$  and  $op_{123} = op_3 \circ op_2 \circ op_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{op_1, op_{12}, op_{123}\}$ 
9:      $x_{aug} += w_i \cdot \text{chain}(x_{aug})$  // Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\hat{\alpha}, \hat{\alpha})$ 
12:  Interpolate with rule  $x_{augmix} = mx_{aug} + (1 - m)x_{orig}$ 
13:  return  $x_{augmix}$ 
14: end function
15:  $x_{augmix1} = \text{AugmentAndMix}(x_{orig})$  //  $x_{augmix1}$  is stochastically generated
16:  $x_{augmix2} = \text{AugmentAndMix}(x_{orig})$  //  $x_{augmix1} \neq x_{augmix2}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(y | x_{orig}), y) + \lambda \text{Jensen-Shannon}(\hat{p}(y | x_{orig}), \hat{p}(y | x_{augmix1}), \hat{p}(y | x_{augmix2}))$ 

```

1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
2. 세 개의 augmentation operation op_1, op_2, op_3 을 샘플링
 $\rightarrow$ 아래와 같은 아홉 개의 augmentation 중 세 개를 복원추출



02 | Augmix

Augment and Mix

❖ Augment and Mix



Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} += w_i \cdot \text{chain}(x_{\text{orig}})$  ▷ Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:  Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:  return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix}1} = \text{AugmentAndMix}(x_{\text{orig}})$  ▷  $x_{\text{augmix}1}$  is stochastically generated
16:  $x_{\text{augmix}2} = \text{AugmentAndMix}(x_{\text{orig}})$  ▷  $x_{\text{augmix}1} \neq x_{\text{augmix}2}$ 

```

1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
2. 세 개의 augmentation operation $\text{op}_1, \text{op}_2, \text{op}_3$ 을 샘플링
3. 연산을 두 개, 세 개로 이어 붙인 $\text{op}_{12}, \text{op}_{123}$ 을 정의
4. $\{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 로부터 한 개를 샘플링 $\rightarrow$ chain

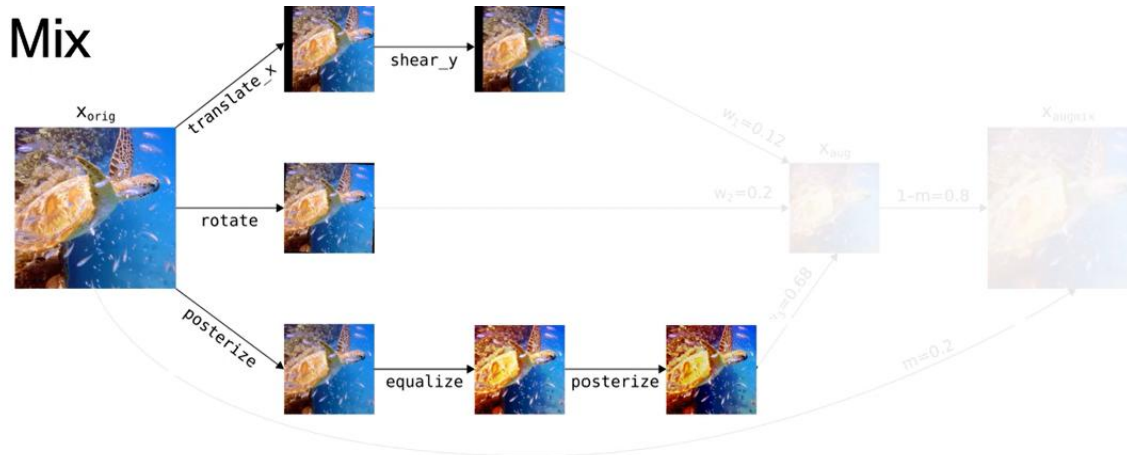
위 그림의 예시를 들어 설명하면 아래와 같음

- 9개의 augmentation 연산 중 translate, shear, rotate 세 개가 샘플링 됨
- $\{\text{translate}, \text{translate} \rightarrow \text{shear}, \text{translate} \rightarrow \text{shear} \rightarrow \text{rotate}\}$ 세 개의 chain 후보가 정의되고 샘플링을 통해 이 중 **translate** $\rightarrow$ **shear**가 chain으로 선택됨

02 | Augmix

Augment and Mix

❖ Augment and Mix



Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} += w_i \cdot \text{chain}(x_{\text{orig}})$  ▷ Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:  Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:  return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix1}} = \text{AugmentAndMix}(x_{\text{orig}})$  ▷  $x_{\text{augmix1}}$  is stochastically generated
16:  $x_{\text{augmix2}} = \text{AugmentAndMix}(x_{\text{orig}})$  ▷  $x_{\text{augmix1}} \neq x_{\text{augmix2}}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(u), x_{\text{augmix1}}) + \lambda \text{KL}(\text{KL}(\hat{p}(u), x_{\text{augmix1}}) \parallel \text{KL}(\hat{p}(u), x_{\text{augmix2}}))$ 

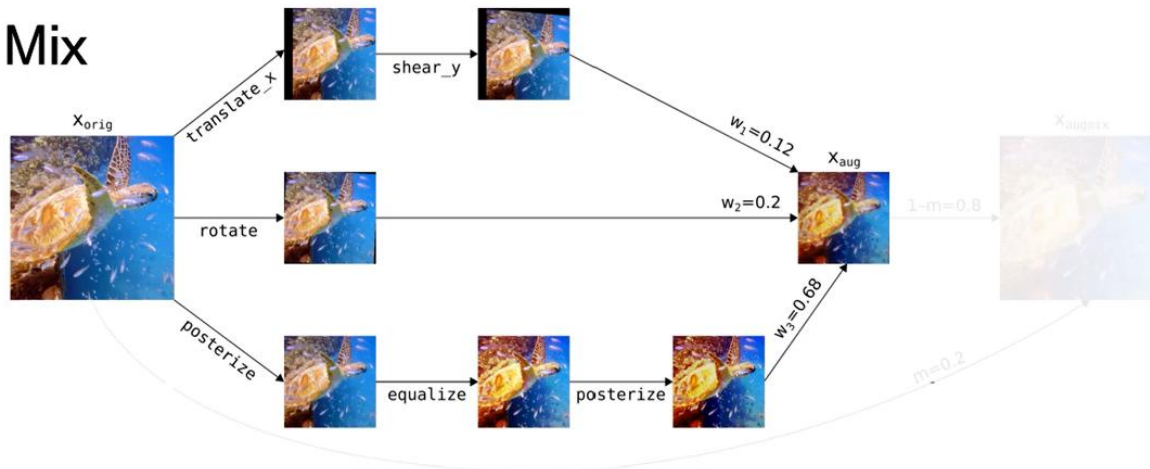
```

1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
2. 세 개의 augmentation operation $\text{op}_1, \text{op}_2, \text{op}_3$ 을 샘플링
3. 연산을 두 개, 세 개로 이어 붙인 $\text{op}_{12}, \text{op}_{123}$ 을 정의
4. $\{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 로부터 한 개를 샘플링 $\rightarrow$ chain
5. 1 ~ 4의 과정을 k 번 반복해서 k 개의 chain을 생성($k = 3$)

02 | Augmix

Augment and Mix

❖ Augment and Mix



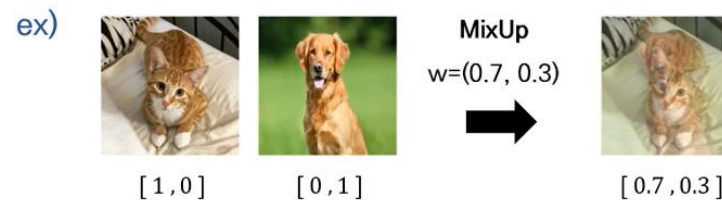
Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}, k=3, \alpha=1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} += w_i \cdot \text{chain}(x_{\text{orig}})$  ▷ Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:  Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:  return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix}1} = \text{AugmentAndMix}(x_{\text{orig}})$  ▷  $x_{\text{augmix}1}$  is stochastically generated
16:  $x_{\text{augmix}2} = \text{AugmentAndMix}(x_{\text{orig}})$  ▷  $x_{\text{augmix}1} \neq x_{\text{augmix}2}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(y | x_{\text{orig}}), y) + \lambda \text{Jensen-Shannon}(\hat{p}(y | x_{\text{orig}}); \hat{p}(y | x_{\text{augmix}1}); \hat{p}(y | x_{\text{augmix}2}))$ 

```

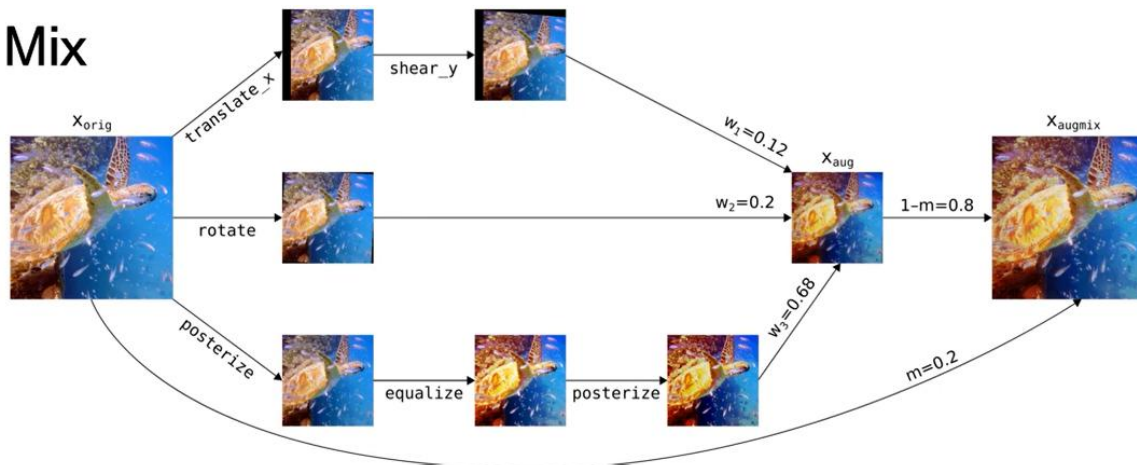
1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
2. 세 개의 augmentation operation $\text{op}_1, \text{op}_2, \text{op}_3$ 을 샘플링
3. 연산을 두 개, 세 개로 이어 붙인 $\text{op}_{12}, \text{op}_{123}$ 을 정의
4. $\{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 로부터 한 개를 샘플링 $\rightarrow$ chain
5. 1 ~ 4의 과정을 k 번 반복해서 k 개의 chain을 생성($k = 3$)
6. 1에서 구한 weight를 이용해 chain들을 MixUp



02 | Augmix

Augment and Mix

❖ Augment and Mix



Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations  $\text{chain} \sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} += w_i \cdot \text{chain}(x_{\text{orig}})$   $\triangleright$  Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:  Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:  return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix1}} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix1}}$  is stochastically generated
16:  $x_{\text{augmix2}} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix1}} \neq x_{\text{augmix2}}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(y | x_{\text{orig}}), y) + \lambda \text{Jensen-Shannon}(\hat{p}(y | x_{\text{orig}}); \hat{p}(y | x_{\text{augmix1}}); \hat{p}(y | x_{\text{augmix2}}))$ 

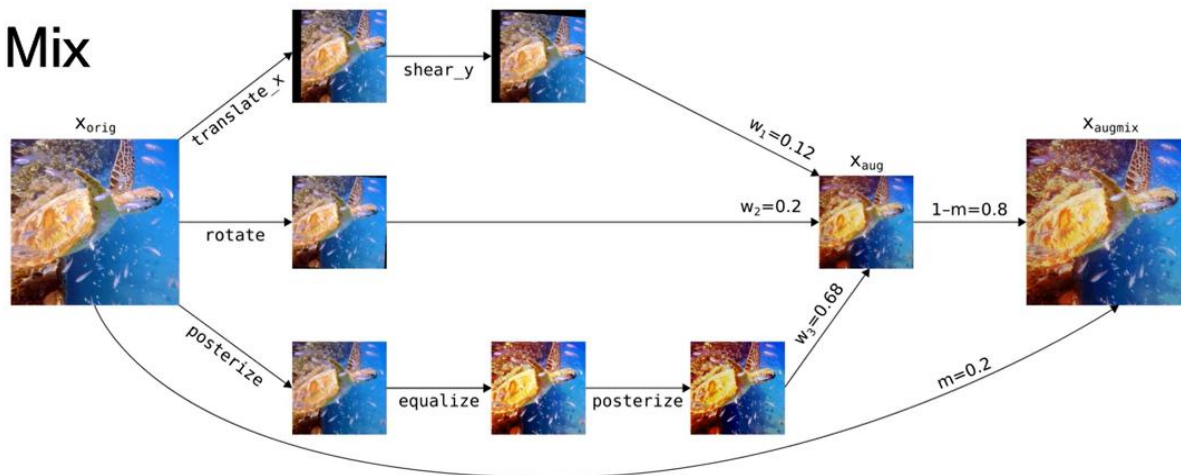
```

1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
2. 세 개의 augmentation operation $\text{op}_1, \text{op}_2, \text{op}_3$ 을 샘플링
3. 연산을 두 개, 세 개로 이어 붙인 $\text{op}_{12}, \text{op}_{123}$ 을 정의
4. $\{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 로부터 한 개를 샘플링 $\rightarrow$ chain
5. 1 ~ 4의 과정을 k 번 반복해서 k 개의 chain을 생성($k = 3$)
6. 1에서 구한 weight를 이용해 chain들을 MixUp
7. $\alpha = 1$ 인 beta 분포로부터 weight 샘플링 $\rightarrow 0.2$
 $\rightarrow$ 0과 1 사이의 scalar값(확률)이 uniform하게 샘플링됨
8. Original data와 6에서 구한 값을 MixUp (skip connection) $\rightarrow x_{\text{augmix}}$

02 | Augmix

Augment and Mix

❖ Augment and Mix



Algorithm AUGMIX Pseudocode

```

1: Input: Model  $\hat{p}$ , Classification Loss  $\mathcal{L}$ , Image  $x_{\text{orig}}$ , Operations  $\mathcal{O} = \{\text{rotate}, \dots, \text{posterize}\}$ 
2: function AugmentAndMix( $x_{\text{orig}}$ ,  $k = 3$ ,  $\alpha = 1$ )
3:   Fill  $x_{\text{aug}}$  with zeros
4:   Sample mixing weights  $(w_1, w_2, \dots, w_k) \sim \text{Dirichlet}(\alpha, \alpha, \dots, \alpha)$ 
5:   for  $i = 1, \dots, k$  do
6:     Sample operations  $\text{op}_1, \text{op}_2, \text{op}_3 \sim \mathcal{O}$ 
7:     Compose operations with varying depth  $\text{op}_{12} = \text{op}_2 \circ \text{op}_1$  and  $\text{op}_{123} = \text{op}_3 \circ \text{op}_2 \circ \text{op}_1$ 
8:     Sample uniformly from one of these operations chain  $\sim \{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 
9:      $x_{\text{aug}} += w_i \cdot \text{chain}(x_{\text{orig}})$   $\triangleright$  Addition is elementwise
10:  end for
11:  Sample weight  $m \sim \text{Beta}(\alpha, \alpha)$ 
12:  Interpolate with rule  $x_{\text{augmix}} = m x_{\text{orig}} + (1 - m) x_{\text{aug}}$ 
13:  return  $x_{\text{augmix}}$ 
14: end function
15:  $x_{\text{augmix1}} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix1}}$  is stochastically generated
16:  $x_{\text{augmix2}} = \text{AugmentAndMix}(x_{\text{orig}})$   $\triangleright x_{\text{augmix1}} \neq x_{\text{augmix2}}$ 
17: Loss Output:  $\mathcal{L}(\hat{p}(y | x_{\text{orig}}), y) + \lambda \text{Jensen-Shannon}(\hat{p}(y | x_{\text{orig}}); \hat{p}(y | x_{\text{augmix1}}); \hat{p}(y | x_{\text{augmix2}}))$ 

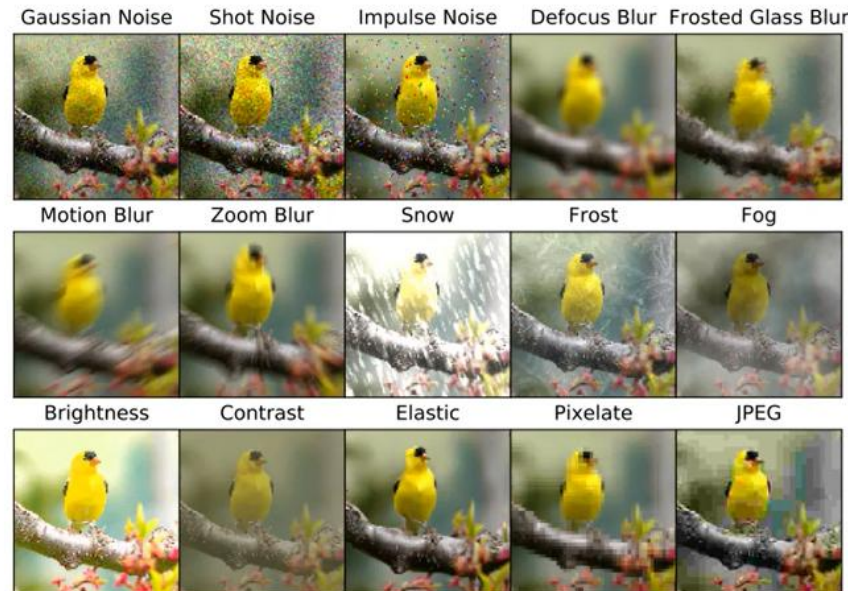
```

1. $k = 3, \alpha = 1$ 인 Dirichlet 분포로부터 weight 샘플링 $\rightarrow (0.12, 0.2, 0.68)$
2. 세 개의 augmentation operation $\text{op}_1, \text{op}_2, \text{op}_3$ 을 샘플링
3. 연산을 두 개, 세 개로 이어 붙인 $\text{op}_{12}, \text{op}_{123}$ 을 정의
4. $\{\text{op}_1, \text{op}_{12}, \text{op}_{123}\}$ 로부터 한 개를 샘플링 $\rightarrow$ chain
5. 1 ~ 4의 과정을 k 번 반복해서 k 개의 chain을 생성($k = 3$)
6. 1에서 구한 weight를 이용해 chain들을 MixUp
7. $\alpha = 1$ 인 beta 분포로부터 weight 샘플링 $\rightarrow 0.2$
8. Original data와 6에서 구한 값을 MixUp (skip connection) $\rightarrow x_{\text{augmix}}$
9. 1~8을 반복해 $x_{\text{augmix1}}, x_{\text{augmix2}}$ 를 생성한 뒤 loss 계산

02 | Augmix

Datasets and Metrics

- CIFAR-10-C, CIFAR-100-C, ImageNet-C
 - ✓ 각 이미지별 15개의 corruption이 존재하며 각 corruption은 5개의 severity(강도)가 존재함
 - ✓ Severity별로 Error를 계산한 뒤 합한 $uCE_c = \sum_{s=1}^5 E_{c,s}$ (Unnormalized Corruption Error) 계산
 - ✓ 15개의 corruption에 대한 uCE_c 를 평균낸 Mean Corruption Error mCE 계산



02 | Augmix

Datasets and Metrics

- CIFAR-10-P, CIFAR-100-P, ImageNet-P
 - ✓ Classifier의 예측안정성(Prediction Stability)를 평가하기 위한 데이터
 - ✓ 각 데이터는 비디오로 이루어졌는데 시간에 따라 perturbation이 변화(이미지가 시간에 따라 기울어짐)
 - ✓ 이상적인 classifier는 perturbation이 변해도 일관적인 예측을 보여줘야 함.
 - ✓ 인접한 두 frame의 예측이 변화하는 것을 “Flip”으로 정의하고 한 영상에 대해서 flip된 비율을 측정



02 | Augmix

CIFAR-10 and CIFAR-100

- CIFAR-10-C와 CIFAR-100-C로 테스트 했을때 Augmix가 가장낮은 error rate를 보임.

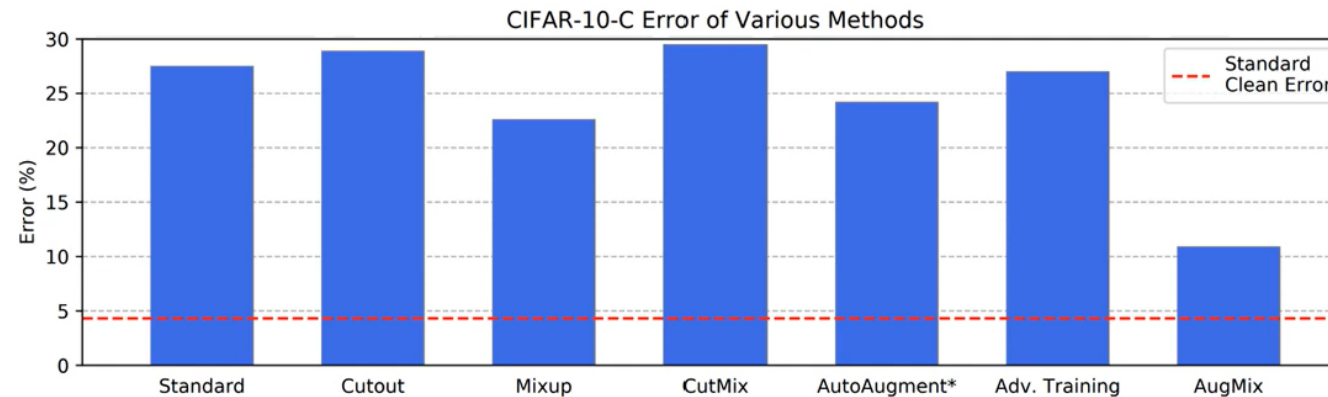


Figure 5: Error rates of various methods on CIFAR-10-C using a ResNeXt backbone. Observe that AUGMIX halves the error rate of prior methods and approaches the clean error rate.

		Standard	Cutout	Mixup	CutMix	AutoAugment*	Adv Training	AUGMIX
CIFAR-10-C	AllConvNet	30.8	32.9	24.6	31.3	29.2	28.1	15.0
	DenseNet	30.7	32.1	24.6	33.5	26.6	27.6	12.7
	WideResNet	26.9	26.8	22.3	27.1	23.9	26.2	11.2
	ResNeXt	27.5	28.9	22.6	29.5	24.2	27.0	10.9
	Mean	29.0	30.2	23.5	30.3	26.0	27.2	12.5
CIFAR-100-C	AllConvNet	56.4	56.8	53.4	56.0	55.1	56.0	42.7
	DenseNet	59.3	59.6	55.4	59.2	53.9	55.2	39.6
	WideResNet	53.3	53.5	50.4	52.9	49.6	55.1	35.9
	ResNeXt	53.4	54.6	51.4	54.1	51.3	54.4	34.9
	Mean	55.6	56.1	52.6	55.5	52.5	55.2	38.3

02 | Augmix ImageNet

- ImageNet-C 결과

Network	Noise				Blur				Weather				Digital				mCE
	Clean	Gauss.	Shot	Impulse	Defocus	Glass	Motion	Zoom	Snow	Frost	Fog	Bright	Contrast	Elastic	Pixel	JPEG	
Standard	23.9	79	80	82	82	90	84	80	86	81	75	65	79	91	77	80	80.6
Patch Uniform	24.5	67	68	70	74	83	81	77	80	74	75	62	77	84	71	71	74.3
AutoAugment* (AA)	22.8	69	68	72	77	83	80	81	79	75	64	56	70	88	57	71	72.7
Random AA*	23.6	70	71	72	80	86	82	81	81	77	72	61	75	88	73	72	76.1
MaxBlur pool	23.0	73	74	76	74	86	78	77	77	72	63	56	68	86	71	71	73.4
SIN	27.2	69	70	70	77	84	76	82	74	75	69	65	69	80	64	77	73.3
AUGMIX	22.4	65	66	67	70	80	66	66	75	72	67	58	58	79	69	69	68.4
AUGMIX+SIN	25.2	61	62	61	69	77	63	72	66	68	63	59	52	74	60	67	64.9

- ImageNet-P 결과

Network	Noise			Blur		Weather		Digital				mFR
	Clean	Gaussian	Shot	Motion	Zoom	Snow	Bright	Translate	Rotate	Tilt	Scale	
Standard	23.9	57	55	62	65	66	65	43	53	57	49	57.2
Patch Uniform	24.5	32	25	50	52	54	57	40	48	49	46	45.3
AutoAugment* (AA)	22.8	50	45	57	68	63	53	40	44	50	46	51.7
Random AA*	23.6	53	46	53	63	59	57	42	48	54	47	52.2
SIN	27.2	53	50	57	72	51	62	43	53	57	53	55.0
MaxBlur pool	23.0	52	51	59	63	57	64	34	43	49	40	51.2
AUGMIX	22.4	46	41	30	47	38	46	25	32	35	33	37.4
AUGMIX+SIN	25.2	45	40	30	54	32	48	27	35	38	39	38.9

03 | Augmix

결론

- 본 논문을 한마디로 정의하면 무엇인가?
 - ✓ 다양한 augmentation을 확률적으로 조합하고, consistency loss를 통해 robustness와 calibration을 동시에 향상시키는 데이터 증강 기법 제안
- 본 논문에서 놀랍고, 새로운 점은 무엇인가?
 - ✓ 복잡한 adversarial training 없이도 corruption robustness를 크게 향상
- 본인이 예상한 바와 어떤 점이 다른가?
 - ✓ 단순 augmentation만으로는 robustness 개선이 제한적일 것이라 예상했으나, mixing + consistency loss로 큰 성능 향상을 달성
- 본 논문과 관련된 본인의 아이디어는 무엇인가?
 - ✓ Dirichlet/Beta sampling 대신 학습 가능한 mixing weight를 도입하여 최적 조합 탐색

Q&A

감사합니다.

01 | 동영상 강의 (추가)

AUGMAX

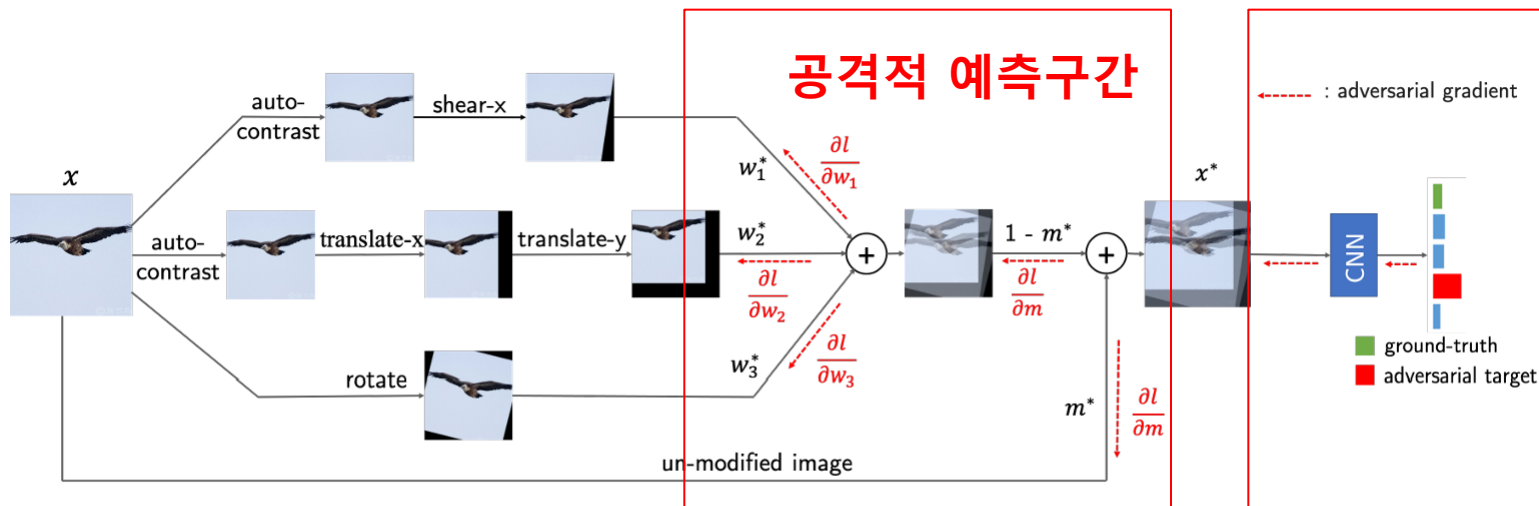


Figure 2: AugMax overview. Black and red arrows represent forward paths and back-propagation paths to generate AugMax images, respectively. In contrast to AugMax, where the mixing parameters m and w are adversarially learned, AugMix randomly samples m and w from predefined distributions (and thus no backpropagation on m and w).

CNN이 틀릴만하게
자동으로 adversarial
Target 조절

이로 인해
작은 변화에 강해짐
일반화 향상
overfitting 감소
robust accuracy 증가

- AUGMIX : 여러 사진 필터를 랜덤하게 섞어서 어떠한 변형에도 안정적 예측을 지향함.
- AUGMAX : 모델이 가장 틀릴 만한 필터 조합을 찾아서 “이것도 맞혀봐” 식의 공격적 예측